

SEMANA 6 – NODE.JS, EXPRESS, JSON Y PETICIONES POST

Actividad: Servidor que recibe datos mediante POST y JSON

<https://github.com/dannysj0214-commits/Programadores-para-la-paz/tree/3afcce4e0edd57e4feff75e453d3a6b197eaae3f/semana6>

1. Preguntas de selección múltiple

Pregunta 1:

¿Qué es JSON?

Respuesta: B. Un formato de intercambio de datos usado entre aplicaciones

Explicación:

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos que es fácil de leer y escribir para los humanos, y fácil de interpretar y generar para las máquinas. Se utiliza ampliamente para transmitir datos entre un servidor y una aplicación web.

Pregunta 2:

¿Qué permite `express.json()` en una aplicación Express?

Respuesta: B. Leer datos enviados en formato JSON en una petición

Explicación:

El middleware `express.json()` es una función incorporada en Express que permite analizar (parsear) las solicitudes entrantes con formato JSON. Esto convierte automáticamente el cuerpo de la petición en un objeto JavaScript accesible a través de `req.body`, facilitando el trabajo con datos estructurados.

Pregunta 3:

¿Dónde se encuentran los datos enviados en una petición POST cuando usamos Express?

Respuesta: B. `req.body`

Explicación:

En Express, los datos enviados en el cuerpo de una petición POST se almacenan en la propiedad `req.body`. Gracias al middleware `express.json()`, estos datos se convierten automáticamente de formato JSON a un objeto JavaScript que puede ser utilizado en la aplicación.

Pregunta 4:

¿Cuál de las siguientes opciones representa correctamente un objeto JSON?

Respuesta: C. `{ "nombre": "Juan" }`

Explicación:

En JSON, las claves deben ir entre comillas dobles y los valores pueden ser cadenas (también entre comillas dobles), números, booleanos, arrays u otros objetos. La opción C es la única que cumple con la sintaxis correcta de JSON. La opción B no tiene comillas en el valor "Juan", y las opciones A y D no tienen el formato correcto de objeto JSON.

2. Código del servidor (server.js)

Archivo: server.js

javascript

```
const express = require('express');
```

```
const app = express();
```

```
// Middleware para parsear JSON
```

```
app.use(express.json());
```

```
// Ruta POST /registro - Recibe nombre y mensaje
```

```
app.post('/registro', (req, res) => {
```

```
  const nombre = req.body.nombre;
```

```
  const mensaje = req.body.mensaje;
```

```
  res.json({
```

```
    estado: "Datos recibidos",
```

```
    nombre: nombre,
```

```
    mensaje: mensaje
```

```
  });
```

```
});
```

```
// Ruta POST /incidencia - Recibe tipo y descripción
```

```
app.post('/incidencia', (req, res) => {
```

```
  const tipo = req.body.tipo;
```

```
const descripcion = req.body.descripcion;

res.json({
  mensaje: "Incidencia registrada",
  tipo: tipo,
  descripcion: descripcion
});
});

// El servidor escucha en el puerto 3000
app.listen(3000, () => {
  console.log('Servidor ejecutándose en puerto 3000');
});
```

Ejecución del servidor:

```
bash
node server.js
```

Rutas disponibles:

Método	Ruta	Descripción
POST	/registro	Recibe nombre y mensaje del usuario
POST	/incidencia	Recibe tipo y descripción de una incidencia

3. Prueba de la API (prueba-api.txt)

Petición POST a /registro

JSON enviado:

```
json
{
  "nombre": "Maria",
```

```
"mensaje": "Hola comunidad"
}
```

Respuesta del servidor:

```
json
{
  "estado": "Datos recibidos",
  "nombre": "Maria",
  "mensaje": "Hola comunidad"
}
```

Petición POST a /incidencia

JSON enviado:

```
json
{
  "tipo": "Iluminación pública",
  "descripcion": "La comunidad reporta que una lámpara del parque no funciona desde hace varios días."
}
```

Respuesta del servidor:

```
json
{
  "mensaje": "Incidencia registrada",
  "tipo": "Iluminación pública",
  "descripcion": "La comunidad reporta que una lámpara del parque no funciona desde hace varios días."
}
```

Explicación del funcionamiento:

1. El servidor utiliza el middleware `express.json()` para interpretar el cuerpo de las peticiones como JSON.
2. En la ruta `/registro`, el servidor extrae los campos `nombre` y `mensaje` del objeto `req.body` y devuelve una respuesta JSON confirmando la recepción.

3. En la ruta /incidencia, el servidor extrae los campos tipo y descripcion y devuelve una respuesta JSON confirmando que la incidencia fue registrada.
 4. El servidor siempre devuelve respuestas en formato JSON, lo que facilita la integración con aplicaciones cliente.
-

4. Ejemplo de incidencia (ejemplo-incidencia.txt)

JSON que podría enviar un ciudadano para reportar una incidencia:

Ejemplo 1 - Problema de iluminación:

```
json
{
  "tipo": "Iluminación pública",
  "descripcion": "La comunidad reporta que una lámpara del parque no funciona desde hace varios días."
}
```

Ejemplo 2 - Problema de bacheo:

```
json
{
  "tipo": "Bacheo",
  "descripcion": "Existe un bache profundo en la calle principal que pone en riesgo a los conductores y peatones."
}
```

Ejemplo 3 - Problema ambiental:

```
json
{
  "tipo": "Ambiente",
  "descripcion": "Acumulación de residuos en la esquina de la carrera 5 con calle 10."
}
```

Ejemplo 4 - Problema de seguridad:

```
json
{
```

```
"tipo": "Seguridad",  
"descripcion": "Falta de señalización vial en el cruce de la avenida principal."  
}
```

Cómo probar la ruta /incidencia:

1. Ejecutar el servidor con `node server.js`
2. Usar Postman, Thunder Client o cURL para enviar una petición POST a `http://localhost:3000/incidencia`
3. Incluir en el cuerpo de la petición un JSON con los campos `tipo` y `descripcion`
4. El servidor responderá confirmando que la incidencia fue registrada

5. Reflexión de la semana (reflexion-semana6.txt)

¿Por qué es importante que las plataformas tecnológicas puedan recibir reportes o información enviada por los ciudadanos de forma estructurada?

Las plataformas tecnológicas que reciben información de los ciudadanos de manera estructurada son fundamentales para la participación ciudadana y la gestión eficiente de las comunidades. Cuando los datos llegan en un formato organizado como JSON, es más fácil procesarlos, almacenarlos y analizarlos, lo que permite tomar decisiones basadas en información real y actualizada.

Recibir reportes estructurados evita la pérdida de información y reduce los errores de interpretación. Por ejemplo, cuando un ciudadano reporta una incidencia con campos claros como "tipo" y "descripcion", el sistema puede clasificar automáticamente el problema y asignarlo al área correspondiente. Esto agiliza la respuesta y mejora la eficiencia de los procesos comunitarios.

Además, la estructuración de los datos permite generar estadísticas y visualizaciones que ayudan a identificar patrones y priorizar acciones. Si muchos ciudadanos reportan problemas en una misma zona, las autoridades pueden enfocar recursos en esa área específica. Esto convierte la información ciudadana en una herramienta poderosa para la planificación y la toma de decisiones.

La estandarización de los datos también facilita la interoperabilidad entre diferentes sistemas. Una plataforma comunitaria puede compartir información con otras aplicaciones o bases de datos gubernamentales, creando un ecosistema de información más completo y útil.

Desde una perspectiva técnica, trabajar con datos estructurados simplifica el desarrollo de aplicaciones y reduce la probabilidad de errores. Los desarrolladores pueden confiar en que los datos recibidos seguirán un formato predecible, lo que acelera el desarrollo y mejora la calidad del software.

En resumen, recibir información de forma estructurada no solo mejora la eficiencia operativa, sino que también fortalece la confianza de los ciudadanos en las plataformas tecnológicas, al demostrar que sus reportes son tomados en serio y procesados de manera profesional y

responsable. Una comunidad que utiliza tecnología para gestionar sus necesidades se vuelve más organizada, participativa y resiliente.

Fecha de entrega: 26/06/2026